Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы трансляции»

ОТЧЕТ
к лабораторной работе №5
на тему:
«ИНТЕРПРЕТАЦИЯ ИСХОДНОГО КОДА»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Описание работы программы 4
3 Ход выполнения программы 5
Вывод 6
Список использованных источников 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

На основе результатов лабораторных работ 1–4 выполнить интерпретацию исходной программы на языке Idris. Необходимо реализовать интерпретатор, использующий результаты лексического, синтаксического и семантического анализа, и обеспечить выполнение программы из файла input.txt.

2 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ

В лабораторной работе был реализован интерпретатор подмножества языка Idris на языке C++. В основу решения положены ранее разработанные этапы лексического, синтаксического и семантического анализа. Это позволило использовать уже подготовленные структуры токенов, логических строк и проверки типов, а затем перейти к вычислению выражений и выполнению программы.

Для представления значений в процессе выполнения введена структура RuntimeValue, поддерживающая целые и вещественные числа, строки, символы, логические значения, кортежи, списки, словари SortedMap и единичное значение. Отдельно описаны структуры и класс IdrisInterpreter, выполняющий сбор определений функций, сопоставление с образцом и вычисление выражений.

Интерпретатор поддерживает вызовы пользовательских функций, конструкции let-in, if-then-else, case-of, do-блоки, работу со строковой конкатенацией, арифметическими и логическими операциями, а также ряд встроенных средств языка, используемых в тестовой программе: putStrLn, show, cast, insert и empty. Для рекурсивной функции forLoop реализовано сопоставление шаблонов.

Выполнение программы начинается с поиска функции main. Затем интерпретатор поочерёдно вычисляет выражения внутри do-блока, накапливает локальные переменные и формирует текстовый вывод программы. При ошибках выполнения формируются отдельные сообщения, которые выводятся в разделе RUNTIME ERRORS.

Программа сохраняет полный цикл предыдущих стадий: сначала печатаются токены, дерево разбора и семантическая таблица, а затем выводятся результаты выполнения интерпретатора. Благодаря такому построению можно проследить путь от исходного текста к реальному вычислению значений.

3 ХОД ВЫПОЛНЕНИЯ ПРОГРАММЫ

Тестирование интерпретатора проводилось на том же файле input.txt. После прохождения стадий анализа программа успешно выполнила функцию main и сформировала ожидаемый результат. Вывод включает строку из функции checkValue, результат сопоставления в processData и последовательность сообщений, генерируемых функцией forLoop.

Для используемого входного файла интерпретатор сформировал следующий вывод: Greater than 5 and Double is positive; Language match!; Iteration: 2; Iteration: 1; Iteration: 0; Done. Это соответствует логике тестовой программы и подтверждает корректность вычисления выражений, сопоставления с образцом и рекурсивных вызовов.

Лексические ошибки, содержащиеся в конце входного файла, были зафиксированы ранее и не повлияли на выполнение корректной части программы.

Таким образом, реализованный интерпретатор способен выполнять подмножество Idris, покрывающее тестовую программу: объявления функций, условные конструкции, рекурсию, списки, кортежи, строки и простые встроенные операции ввода-вывода. Вывод продемонстрирован ниже, на рисунке 3.1.

```
========================================================================
INTERPRETER OUTPUT
========================================================================
Greater than 5 and Double is positive
Language match!
Iteration: 2
Iteration: 1
Iteration: 0
Done
```

Рисунок 3.1 – Вывод интерпретатора

ВЫВОД

В ходе выполнения лабораторной работы разработан интерпретатор для подмножества языка Idris, завершающий полный цикл трансляции исходного кода: лексический анализ, синтаксический разбор, семантический контроль и непосредственное выполнение программы. Интерпретатор построен на основе ранее созданных компонентов и интегрирует их в единую систему. Реализована структура RuntimeValue, поддерживающая целые и вещественные числа, строки, символы, логические значения, кортежи, списки, словари SortedMap и единичное значение. В классе IdrisInterpreter реализованы: сбор определений функций с поддержкой нескольких альтернативных клауз (сопоставление с образцом), вычисление выражений с учётом приоритетов операторов, обработка управляющих конструкций if-then-else, let-in, case-of и do-блоков. Для встроенных функций putStrLn, show, cast, insert и предопределённого значения empty заданы специальные семантические правила, а рекурсивные функции (например, forLoop) корректно работают благодаря сопоставлению шаблонов с конструкторами S и Z. Тестирование на файле input.txt подтвердило, что интерпретатор выдаёт ожидаемые строки: результат проверки значения, сопоставления в case, последовательность итераций рекурсивного цикла. Таким образом, все цели лабораторной работы выполнены: разработанный интерпретатор способен выполнять программы на подмножестве Idris, используя результаты полного синтаксического и семантического анализа.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Скотт, М. Л. Программирование языков программирования: ДМК Пресс, 2012. – 936 с.

[2] Пратт, Т. Языки программирования: разработка и реализации: Питер, 2002. – 688 с.

[3] Документация по Idris [Электронный ресурс]. – Режим доступа: https://docs.idris-lang.org/en/latest. – Дата доступа: 11.02.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация интерпретатора

```
class IdrisInterpreter
{
private:
    const vector<Token> &tokens;
    const vector<LogicalLine> &lines;
    map<string, vector<FunctionClause>> functions;
    vector<string> runtimeOutput;
    vector<string> runtimeErrors;

    static bool hasLexicalError(const LogicalLine &line)
    {
        for (const auto &token : line.tokens)
        {
            if (token.kind == TokenKind::Error)
            {
                return true;
            }
        }
        return false;
    }

    static bool isFullyWrapped(const vector<Token> &slice, const string
&openToken, const string &closeToken)
    {
        if (slice.size() < 2U || slice.front().value != openToken ||
slice.back().value != closeToken)
        {
            return false;
        }

        int depth = 0;
        for (size_t i = 0; i < slice.size(); ++i)
        {
            if (slice[i].value == openToken)
            {
                depth++;
            }
            else if (slice[i].value == closeToken)
            {
                depth--;
                if (depth == 0 && i + 1 != slice.size())
                {
                    return false;
                }
            }
        }

        return depth == 0;
    }

    static int findTopLevelToken(const vector<Token> &slice, const set<string>
&targets, bool searchFromRight = false)
    {
        int parenDepth = 0;
        int bracketDepth = 0;
```

```
        if (!searchFromRight)
        {
            for (size_t i = 0; i < slice.size(); ++i)
            {
                const string &value = slice[i].value;
                if (value == "(")
                {
                    parenDepth++;
                    continue;
                }
                if (value == ")")
                {
                    parenDepth--;
                    continue;
                }
                if (value == "[")
                {
                    bracketDepth++;
                    continue;
                }
                if (value == "]")
                {
                    bracketDepth--;
                    continue;
                }
                if (parenDepth == 0 && bracketDepth == 0 && targets.count(value)
!= 0U)
                {
                    return static_cast<int>(i);
                }
            }
            return -1;
        }

        for (int i = static_cast<int>(slice.size()) - 1; i >= 0; --i)
        {
            const string &value = slice[static_cast<size_t>(i)].value;
            if (value == ")")
            {
                parenDepth++;
                continue;
            }
            if (value == "(")
            {
                parenDepth--;
                continue;
            }
            if (value == "]")
            {
                bracketDepth++;
                continue;
            }
            if (value == "[")
            {
                bracketDepth--;
                continue;
            }
            if (parenDepth == 0 && bracketDepth == 0 && targets.count(value) !=
0U)
            {
                return i;
            }
        }
```

```
        return -1;
    }

    static vector<vector<Token>> splitTopLevel(const vector<Token> &slice,
const string &delimiter)
    {
        vector<vector<Token>> parts;
        vector<Token> current;
        int parenDepth = 0;
        int bracketDepth = 0;

        for (const auto &token : slice)
        {
            if (token.value == "(")
            {
                parenDepth++;
            }
            else if (token.value == ")")
            {
                parenDepth--;
            }
            else if (token.value == "[")
            {
                bracketDepth++;
            }
            else if (token.value == "]")
            {
                bracketDepth--;
            }

            if (parenDepth == 0 && bracketDepth == 0 && token.value ==
delimiter)
            {
                parts.push_back(current);
                current.clear();
                continue;
            }

            current.push_back(token);
        }

        parts.push_back(current);
        return parts;
    }

    static size_t consumeTerm(const vector<Token> &slice, size_t start)
    {
        if (start >= slice.size())
        {
            return start;
        }

        const string &value = slice[start].value;
        if (value == "(" || value == "[")
        {
            const string closeToken = value == "(" ? ")" : "]";
            int depth = 0;
            for (size_t i = start; i < slice.size(); ++i)
            {
                if (slice[i].value == value)
                {
                    depth++;
                }
```

```
                else if (slice[i].value == closeToken)
                {
                    depth--;
                    if (depth == 0)
                    {
                        return i + 1;
                    }
                }
            }
            return slice.size();
        }

        return start + 1;
    }

    static vector<vector<Token>> splitApplicationTerms(const vector<Token>
&slice)
    {
        vector<vector<Token>> terms;
        size_t index = 0;
        while (index < slice.size())
        {
            size_t next = consumeTerm(slice, index);
            if (next <= index)
            {
                break;
            }
            terms.emplace_back(slice.begin()    +    static_cast<long>(index),
slice.begin() + static_cast<long>(next));
            index = next;
        }
        return terms;
    }

    static vector<Token> flattenLines(size_t start, size_t end, const
vector<LogicalLine> &allLines)
    {
        vector<Token> merged;
        for (size_t i = start; i < end; ++i)
        {
            merged.insert(merged.end(),            allLines[i].tokens.begin(),
allLines[i].tokens.end());
        }
        return merged;
    }

    size_t findBlockEnd(size_t start, int parentIndent) const
    {
        size_t index = start;
        while (index < lines.size() && lines[index].indent > parentIndent)
        {
            ++index;
        }
        return index;
    }

    void addRuntimeError(const vector<Token> &context, const string &message)
    {
        if (!context.empty())
        {
            runtimeErrors.push_back("Line " + to_string(context.front().line)
+ ", column " + to_string(context.front().column) + ": " + message);
        }
        else
```

```
        {
            runtimeErrors.push_back(message);
        }
    }

    double asNumber(const RuntimeValue &value) const
    {
        if (value.kind == RuntimeValue::Kind::Int)
        {
            return static_cast<double>(value.intValue);
        }
        if (value.kind == RuntimeValue::Kind::Double)
        {
            return value.doubleValue;
        }
        return 0.0;
    }

    bool equals(const RuntimeValue &left, const RuntimeValue &right) const
    {
        if (left.kind != right.kind)
        {
            if ((left.kind == RuntimeValue::Kind::Int || left.kind ==
RuntimeValue::Kind::Double) &&
                (right.kind == RuntimeValue::Kind::Int || right.kind ==
RuntimeValue::Kind::Double))
            {
                return asNumber(left) == asNumber(right);
            }
            return false;
        }

        switch (left.kind)
        {
        case RuntimeValue::Kind::Int:
            return left.intValue == right.intValue;
        case RuntimeValue::Kind::Double:
            return left.doubleValue == right.doubleValue;
        case RuntimeValue::Kind::String:
            return left.stringValue == right.stringValue;
        case RuntimeValue::Kind::Char:
            return left.charValue == right.charValue;
        case RuntimeValue::Kind::Bool:
            return left.boolValue == right.boolValue;
        case RuntimeValue::Kind::Unit:
            return true;
        default:
            return left.toString() == right.toString();
        }
    }

    bool matchPattern(const vector<Token> &pattern, const RuntimeValue &value,
map<string, RuntimeValue> &bindings)
    {
        if (pattern.empty())
        {
            return false;
        }

        if (isFullyWrapped(pattern, "(", ")"))
        {
            vector<Token> inner(pattern.begin() + 1, pattern.end() - 1);
            if (inner.empty())
            {
```

```
                return value.kind == RuntimeValue::Kind::Unit;
            }

            auto parts = splitTopLevel(inner, ",");
            if (parts.size() > 1U)
            {
                if     (value.kind     !=     RuntimeValue::Kind::Tuple     ||
value.elements.size() != parts.size())
                {
                    return false;
                }

                for (size_t i = 0; i < parts.size(); ++i)
                {
                    if (!matchPattern(parts[i], value.elements[i], bindings))
                    {
                        return false;
                    }
                }
                return true;
            }

            return matchPattern(inner, value, bindings);
        }

        auto terms = splitApplicationTerms(pattern);
        if   (terms.size()   ==   2U   &&   terms.front().size()   ==   1U   &&
terms.front().front().value == "S")
        {
            if (value.kind != RuntimeValue::Kind::Int || value.intValue <= 0)
            {
                return false;
            }
            RuntimeValue predecessor = RuntimeValue::makeInt(value.intValue -
1);
            return matchPattern(terms[1], predecessor, bindings);
        }

        if (pattern.size() == 1U)
        {
            const Token &token = pattern.front();
            if (token.value == "_")
            {
                return true;
            }
            if (token.value == "Z")
            {
                return value.kind == RuntimeValue::Kind::Int && value.intValue
== 0;
            }
            if (token.kind == TokenKind::StringLiteral)
            {
                return    value.kind    ==    RuntimeValue::Kind::String    &&
value.stringValue == token.value;
            }
            if (token.kind == TokenKind::CharLiteral)
            {
                return    value.kind    ==    RuntimeValue::Kind::Char    &&
value.charValue == token.value.front();
            }
            if (token.kind == TokenKind::Number)
            {
                if (token.value.find('.') != string::npos)
                {
```

```
                    return value.kind == RuntimeValue::Kind::Double &&
value.doubleValue == stod(token.value);
                }
                return value.kind == RuntimeValue::Kind::Int && value.intValue
== stoi(token.value);
            }
            if (token.kind == TokenKind::Identifier || token.kind ==
TokenKind::TypeName)
            {
                if (!token.value.empty() && isupper(static_cast<unsigned
char>(token.value.front())))
                {
                    return false;
                }
                bindings[token.value] = value;
                return true;
            }
        }

        return false;
    }

    RuntimeValue callFunction(const string &name, const vector<RuntimeValue>
&arguments)
    {
        if (name == "putStrLn")
        {
            if (arguments.size() != 1U)
            {
                runtimeErrors.push_back("putStrLn expects one argument");
                return RuntimeValue::makeUnit();
            }
            runtimeOutput.push_back(arguments[0].toString());
            return RuntimeValue::makeUnit();
        }

        if (name == "show")
        {
            if (arguments.size() != 1U)
            {
                runtimeErrors.push_back("show expects one argument");
                return RuntimeValue::makeString("");
            }
            return RuntimeValue::makeString(arguments[0].toString());
        }

        if (name == "cast")
        {
            if (arguments.size() != 1U)
            {
                runtimeErrors.push_back("cast expects one argument");
                return RuntimeValue::makeInt(0);
            }
            return arguments[0];
        }

        if (name == "insert")
        {
            if (arguments.size() != 3U)
            {
                runtimeErrors.push_back("insert expects three arguments");
                return RuntimeValue::makeSortedMap({});
            }
```

```
            map<string, RuntimeValue> result = arguments[2].mapValue;
            result[arguments[0].toString()] = arguments[1];
            return RuntimeValue::makeSortedMap(result);
        }

        if (name == "empty")
        {
            return RuntimeValue::makeSortedMap({});
        }

        auto functionIt = functions.find(name);
        if (functionIt == functions.end())
        {
            runtimeErrors.push_back("unknown function: " + name);
            return RuntimeValue::makeUnit();
        }

        for (const auto &clause : functionIt->second)
        {
            if (clause.patterns.size() != arguments.size())
            {
                continue;
            }

            map<string, RuntimeValue> env;
            bool matched = true;
            for (size_t i = 0; i < clause.patterns.size(); ++i)
            {
                if (!matchPattern(clause.patterns[i], arguments[i], env))
                {
                    matched = false;
                    break;
                }
            }

            if (!matched)
            {
                continue;
            }

            if (!clause.rhsInline.empty())
            {
                if (clause.rhsInline.front().value == "do")
                {
                    return                 evaluateDoBlock(clause.nestedStart,
clause.nestedEnd, env);
                }
                return evaluateExpression(clause.rhsInline, env);
            }

            if (clause.nestedStart < clause.nestedEnd)
            {
                if        (!lines[clause.nestedStart].tokens.empty()        &&
lines[clause.nestedStart].tokens.front().value == "case")
                {
                    return               evaluateCaseBlock(clause.nestedStart,
clause.nestedEnd, env);
                }
                if        (!lines[clause.nestedStart].tokens.empty()        &&
lines[clause.nestedStart].tokens.front().value == "do")
                {
                    return     evaluateDoBlock(clause.nestedStart     +     1,
clause.nestedEnd, env);
                }
```

```
                return     evaluateExpression(flattenLines(clause.nestedStart,
clause.nestedEnd, lines), env);
            }

            return RuntimeValue::makeUnit();
        }

        runtimeErrors.push_back("no matching clause for function: " + name);
        return RuntimeValue::makeUnit();
    }

    RuntimeValue   evaluateAtom(const   vector<Token>   &slice,   map<string,
RuntimeValue> &env)
    {
        if (slice.empty())
        {
            return RuntimeValue::makeUnit();
        }

        if (isFullyWrapped(slice, "(", ")"))
        {
            vector<Token> inner(slice.begin() + 1, slice.end() - 1);
            if (inner.empty())
            {
                return RuntimeValue::makeUnit();
            }

            auto parts = splitTopLevel(inner, ",");
            if (parts.size() > 1U)
            {
                vector<RuntimeValue> values;
                for (const auto &part : parts)
                {
                    values.push_back(evaluateExpression(part, env));
                }
                return RuntimeValue::makeTuple(values);
            }
            return evaluateExpression(inner, env);
        }

        if (isFullyWrapped(slice, "[", "]"))
        {
            vector<Token> inner(slice.begin() + 1, slice.end() - 1);
            vector<RuntimeValue> values;
            if (!inner.empty())
            {
                auto parts = splitTopLevel(inner, ",");
                for (const auto &part : parts)
                {
                    values.push_back(evaluateExpression(part, env));
                }
            }
            return RuntimeValue::makeList(values);
        }

        if (slice.size() == 1U)
        {
            const Token &token = slice.front();
            if (token.kind == TokenKind::Number)
            {
                if (token.value.find('.') != string::npos)
                {
                    return RuntimeValue::makeDouble(stod(token.value));
                }
```

```
                try
                {
                    return RuntimeValue::makeInt(stoi(token.value));
                }
                catch (const out_of_range &)
                {
                    return RuntimeValue::makeString(token.value);
                }
            }
            if (token.kind == TokenKind::StringLiteral)
            {
                return RuntimeValue::makeString(token.value);
            }
            if (token.kind == TokenKind::CharLiteral)
            {
                return  RuntimeValue::makeChar(token.value.empty()  ?  '\0'  :
token.value.front());
            }
            if   (token.kind   ==   TokenKind::Identifier   ||   token.kind   ==
TokenKind::TypeName)
            {
                auto envIt = env.find(token.value);
                if (envIt != env.end())
                {
                    return envIt->second;
                }
                if (token.value == "empty")
                {
                    return RuntimeValue::makeSortedMap({});
                }
                auto fnIt = functions.find(token.value);
                if (fnIt != functions.end() && !fnIt->second.empty() && fnIt-
>second.front().patterns.empty())
                {
                    return callFunction(token.value, {});
                }
            }
        }

        addRuntimeError(slice, "unable to evaluate atom");
        return RuntimeValue::makeUnit();
    }

    RuntimeValue  evaluateExpression(const  vector<Token>  &slice,  map<string,
RuntimeValue> &env)
    {
        if (slice.empty())
        {
            return RuntimeValue::makeUnit();
        }

        if (slice.front().value == "if")
        {
            int thenIndex = findTopLevelToken(slice, {"then"});
            int elseIndex = findTopLevelToken(slice, {"else"});
            vector<Token> conditionTokens(slice.begin() + 1, slice.begin() +
thenIndex);
            vector<Token>   thenTokens(slice.begin()   +   thenIndex   +   1,
slice.begin() + elseIndex);
            vector<Token>   elseTokens(slice.begin()   +   elseIndex   +   1,
slice.end());

            RuntimeValue condition = evaluateExpression(conditionTokens, env);
            if (condition.kind != RuntimeValue::Kind::Bool)
```

```
            {
                addRuntimeError(conditionTokens,   "if   condition   is   not
boolean");
                return RuntimeValue::makeUnit();
            }

            return condition.boolValue ? evaluateExpression(thenTokens, env) :
evaluateExpression(elseTokens, env);
        }

        if (slice.front().value == "let")
        {
            int inIndex = findTopLevelToken(slice, {"in"});
            if (inIndex < 0)
            {
                addRuntimeError(slice, "let expression must contain in");
                return RuntimeValue::makeUnit();
            }

            vector<Token> binding(slice.begin() + 1, slice.begin() + inIndex);
            vector<Token> body(slice.begin() + inIndex + 1, slice.end());
            map<string, RuntimeValue> nestedEnv = env;
            evaluateLetBinding(binding, nestedEnv);
            return evaluateExpression(body, nestedEnv);
        }

        static const vector<set<string>> precedence = {
            {"&&", "||"},
            {">", "<", ">=", "<=", "==", "/="},
            {"++"},
            {"+", "-"},
            {"*", "/"}};

        for (const auto &level : precedence)
        {
            int opIndex = findTopLevelToken(slice, level, true);
            if (opIndex > 0 && static_cast<size_t>(opIndex + 1) < slice.size())
            {
                vector<Token>   leftTokens(slice.begin(),   slice.begin()   +
opIndex);
                vector<Token>   rightTokens(slice.begin()   +   opIndex   +   1,
slice.end());
                RuntimeValue left = evaluateExpression(leftTokens, env);
                RuntimeValue right = evaluateExpression(rightTokens, env);
                const string op = slice[static_cast<size_t>(opIndex)].value;

                if (op == "&&")
                {
                    return      RuntimeValue::makeBool(left.boolValue      &&
right.boolValue);
                }
                if (op == "||")
                {
                    return      RuntimeValue::makeBool(left.boolValue      ||
right.boolValue);
                }
                if (op == "++")
                {
                    return     RuntimeValue::makeString(left.toString()     +
right.toString());
                }
                if (op == ">")
                {
```

```
                    return       RuntimeValue::makeBool(asNumber(left)       >
asNumber(right));
                }
                if (op == "<")
                {
                    return       RuntimeValue::makeBool(asNumber(left)       <
asNumber(right));
                }
                if (op == ">=")
                {
                    return      RuntimeValue::makeBool(asNumber(left)      >=
asNumber(right));
                }
                if (op == "<=")
                {
                    return      RuntimeValue::makeBool(asNumber(left)      <=
asNumber(right));
                }
                if (op == "==" || op == "/=")
                {
                    bool result = equals(left, right);
                    return  RuntimeValue::makeBool(op  ==  "=="  ?  result  :
!result);
                }
                if (op == "+")
                {
                    if (left.kind == RuntimeValue::Kind::Double || right.kind
== RuntimeValue::Kind::Double)
                    {
                        return    RuntimeValue::makeDouble(asNumber(left)    +
asNumber(right));
                    }
                    return
RuntimeValue::makeInt(static_cast<int>(asNumber(left) + asNumber(right)));
                }
                if (op == "-")
                {
                    if (left.kind == RuntimeValue::Kind::Double || right.kind
== RuntimeValue::Kind::Double)
                    {
                        return    RuntimeValue::makeDouble(asNumber(left)    -
asNumber(right));
                    }
                    return
RuntimeValue::makeInt(static_cast<int>(asNumber(left) - asNumber(right)));
                }
                if (op == "*")
                {
                    if (left.kind == RuntimeValue::Kind::Double || right.kind
== RuntimeValue::Kind::Double)
                    {
                        return    RuntimeValue::makeDouble(asNumber(left)    *
asNumber(right));
                    }
                    return
RuntimeValue::makeInt(static_cast<int>(asNumber(left) * asNumber(right)));
                }
                if (op == "/")
                {
                    return      RuntimeValue::makeDouble(asNumber(left)      /
asNumber(right));
                }
            }
        }
```

```
        auto terms = splitApplicationTerms(slice);
        if (terms.size() > 1U && terms.front().size() == 1U)
        {
            string functionName = terms.front().front().value;
            vector<RuntimeValue> arguments;
            for (size_t i = 1; i < terms.size(); ++i)
            {
                arguments.push_back(evaluateExpression(terms[i], env));
            }
            return callFunction(functionName, arguments);
        }

        return evaluateAtom(slice, env);
    }

    void evaluateLetBinding(const vector<Token> &binding, map<string,
RuntimeValue> &env)
    {
        int equalIndex = findTopLevelToken(binding, {"="});
        if (equalIndex < 0 || binding.empty())
        {
            addRuntimeError(binding, "invalid let binding");
            return;
        }

        int colonIndex = findTopLevelToken(vector<Token>(binding.begin(),
binding.begin() + equalIndex), {":"});
        int valueStart = equalIndex + 1;
        string name = binding.front().value;
        vector<Token> valueTokens(binding.begin() + valueStart,
binding.end());
        env[name] = evaluateExpression(valueTokens, env);

        (void)colonIndex;
    }

    RuntimeValue evaluateCaseBlock(size_t start, size_t end, map<string,
RuntimeValue> &env)
    {
        const auto &caseLine = lines[start];
        int ofIndex = findTopLevelToken(caseLine.tokens, {"of"});
        if (ofIndex < 0)
        {
            addRuntimeError(caseLine.tokens, "case expression must contain
of");
            return RuntimeValue::makeUnit();
        }

        vector<Token> subjectTokens(caseLine.tokens.begin() + 1,
caseLine.tokens.begin() + ofIndex);
        RuntimeValue subject = evaluateExpression(subjectTokens, env);

        size_t branchIndex = start + 1;
        while (branchIndex < end)
        {
            const auto &branchLine = lines[branchIndex];
            if (branchLine.indent <= caseLine.indent)
            {
                break;
            }

            size_t branchEnd = branchIndex + 1;
```

```
            while (branchEnd < end && lines[branchEnd].indent > branchLine.indent)
            {
                ++branchEnd;
            }

            int arrowIndex = findTopLevelToken(branchLine.tokens, {"=>"});
            if (arrowIndex < 0)
            {
                addRuntimeError(branchLine.tokens, "case branch must contain =>");
                branchIndex = branchEnd;
                continue;
            }

            vector<Token> patternTokens(branchLine.tokens.begin(), branchLine.tokens.begin() + arrowIndex);
            map<string, RuntimeValue> branchEnv = env;
            if (matchPattern(patternTokens, subject, branchEnv))
            {
                vector<Token> valueTokens(branchLine.tokens.begin() + arrowIndex + 1, branchLine.tokens.end());
                if (!valueTokens.empty())
                {
                    return evaluateExpression(valueTokens, branchEnv);
                }
                return evaluateExpression(flattenLines(branchIndex + 1, branchEnd, lines), branchEnv);
            }

            branchIndex = branchEnd;
        }

        addRuntimeError(caseLine.tokens, "no matching case branch");
        return RuntimeValue::makeUnit();
    }

    RuntimeValue evaluateDoBlock(size_t start, size_t end, map<string, RuntimeValue> &env)
    {
        RuntimeValue last = RuntimeValue::makeUnit();
        size_t statementIndex = start;

        while (statementIndex < end)
        {
            const auto &line = lines[statementIndex];
            size_t statementEnd = statementIndex + 1;
            while (statementEnd < end && lines[statementEnd].indent > line.indent)
            {
                ++statementEnd;
            }

            if (!line.tokens.empty() && line.tokens.front().value == "let")
            {
                evaluateLetBinding(vector<Token>(line.tokens.begin() + 1, line.tokens.end()), env);
            }
            else if (!line.tokens.empty() && line.tokens.front().value == "case")
            {
                last = evaluateCaseBlock(statementIndex, statementEnd, env);
            }
            else
```

```
            {
                vector<Token> expressionTokens = line.tokens;
                if (statementEnd > statementIndex + 1)
                {
                    auto nested = flattenLines(statementIndex + 1,
statementEnd, lines);
                    expressionTokens.insert(expressionTokens.end(),
nested.begin(), nested.end());
                }
                last = evaluateExpression(expressionTokens, env);
            }

            statementIndex = statementEnd;
        }

        return last;
    }

    void collectFunctions()
    {
        size_t index = 0;
        while (index < lines.size())
        {
            const auto &line = lines[index];
            if (line.indent != 1 || hasLexicalError(line))
            {
                ++index;
                continue;
            }

            int equalIndex = findTopLevelToken(line.tokens, {"="});
            if (equalIndex <= 0)
            {
                ++index;
                continue;
            }

            FunctionClause clause;
            clause.name = line.tokens.front().value;
            clause.line = line.lineNumber;

            vector<Token>       lhsTokens(line.tokens.begin()       +       1,
line.tokens.begin() + equalIndex);
            clause.patterns = splitApplicationTerms(lhsTokens);
            clause.rhsInline = vector<Token>(line.tokens.begin() + equalIndex
+ 1, line.tokens.end());
            clause.nestedStart = index + 1;
            clause.nestedEnd = findBlockEnd(index + 1, line.indent);
            functions[clause.name].push_back(clause);
            index = clause.nestedEnd;
        }
    }

public:
    IdrisInterpreter(const vector<Token> &allTokens, const vector<LogicalLine>
&logicalLines)
        : tokens(allTokens), lines(logicalLines) {}

    void interpret()
    {
        collectFunctions();
        if (functions.find("main") == functions.end())
        {
            runtimeErrors.push_back("main function not found");
```

```
            return;
        }

        vector<RuntimeValue> noArgs;
        (void)callFunction("main", noArgs);
    }

    const vector<string> &getOutput() const
    {
        return runtimeOutput;
    }

    const vector<string> &getErrors() const
    {
        return runtimeErrors;
    }
};

string readInputFile()
{
    const vector<string> candidates = {
        "../../lab4/build/input.txt",
        "../lab4/build/input.txt",
        "lab4/build/input.txt",
        "input.txt",
        "build/input.txt",
        "../../lab5/build/input.txt"};

    for (const auto &path : candidates)
    {
        ifstream input(path);
        if (!input.is_open())
        {
            continue;
        }

        stringstream buffer;
        buffer << input.rdbuf();
        return buffer.str();
    }

    return {};
}

string tokenKindToString(TokenKind kind)
{
    switch (kind)
    {
    case TokenKind::Reserved:
        return "Reserved";
    case TokenKind::Identifier:
        return "Identifier";
    case TokenKind::TypeName:
        return "Type";
    case TokenKind::Number:
        return "Number";
    case TokenKind::StringLiteral:
        return "String";
    case TokenKind::CharLiteral:
        return "Char";
    case TokenKind::Operator:
        return "Operator";
    case TokenKind::Separator:
        return "Separator";
```

```
    case TokenKind::Newline:
        return "Newline";
    case TokenKind::EndOfFile:
        return "EOF";
    case TokenKind::Error:
        return "Error";
    }
    return "Unknown";
}

void printTree(const shared_ptr<AstNode> &node, const string &prefix = "", bool
isLast = true, bool isRoot = true)
{
    if (!node)
    {
        return;
    }

    if (isRoot)
    {
        cout << node->label << '\n';
    }
    else
    {
        cout << prefix;
        cout << (isLast ? "└── " : "├── ");
        cout << node->label << '\n';
    }

    for (size_t i = 0; i < node->children.size(); ++i)
    {
        bool childIsLast = i + 1 == node->children.size();
        string childPrefix = isRoot ? "" : prefix + (isLast ? "    " : "│   ");
        printTree(node->children[i], childPrefix, childIsLast, false);
    }
}

void printTokens(const vector<Token> &tokens)
{
    cout << string(70, '=') << '\n';
    cout << "TOKEN TABLE\n";
    cout << string(70, '=') << '\n';
    cout << left << setw(14) << "TYPE" << setw(28) << "VALUE" << setw(8) <<
"LINE" << setw(8) << "COLUMN" << '\n';
    cout << string(70, '-') << '\n';

    for (const auto &token : tokens)
    {
        cout << left << setw(14) << tokenKindToString(token.kind) << setw(28)
             << (token.kind == TokenKind::StringLiteral ? "\"" + token.value +
"\"" : token.kind == TokenKind::CharLiteral ? "'" + token.value + "'" :
token.value)
             << setw(8) << token.line << setw(8) << token.column << '\n';
    }
}

void printErrors(const vector<Token> &tokens, const vector<string>
&syntaxErrors)
{
    vector<string> lexicalErrors;
    for (const auto &token : tokens)
    {
        if (token.kind == TokenKind::Error)
        {
```

```
            lexicalErrors.push_back("Line " + to_string(token.line) + ", column
" + to_string(token.column) + ": " + token.value);
        }
    }

    cout << "\n" << string(70, '=') << '\n';
    cout << "LEXICAL ERRORS\n";
    cout << string(70, '=') << '\n';
    if (lexicalErrors.empty())
    {
        cout << "No lexical errors found.\n";
    }
    else
    {
        for (const auto &error : lexicalErrors)
        {
            cout << error << '\n';
        }
    }

    cout << "\n" << string(70, '=') << '\n';
    cout << "SYNTAX ERRORS\n";
    cout << string(70, '=') << '\n';
    if (syntaxErrors.empty())
    {
        cout << "No syntax errors found.\n";
    }
    else
    {
        for (const auto &error : syntaxErrors)
        {
            cout << error << '\n';
        }
    }
}

void printSemanticSymbols(const vector<SymbolEntry> &symbols)
{
    const int totalWidth = 96;
    const int scopeWidth = 18;
    const int nameWidth = 18;
    const int typeWidth = 48;
    const int lineWidth = 8;

    cout << "\n" << string(totalWidth, '=') << '\n';
    cout << "SEMANTIC SYMBOL TABLE\n";
    cout << string(totalWidth, '=') << '\n';
    cout << left << setw(scopeWidth) << "SCOPE"
         << setw(nameWidth) << "NAME"
         << setw(typeWidth) << "TYPE"
         << setw(lineWidth) << "LINE" << '\n';
    cout << string(totalWidth, '-') << '\n';

    for (const auto &entry : symbols)
    {
        cout << left << setw(scopeWidth) << entry.scope
             << setw(nameWidth) << entry.name
             << setw(typeWidth) << entry.type
             << setw(lineWidth) << entry.line << '\n';
    }
}

void printSemanticErrors(const vector<string> &semanticErrors)
{
```

```
    cout << "\n" << string(70, '=') << '\n';
    cout << "SEMANTIC ERRORS\n";
    cout << string(70, '=') << '\n';
    if (semanticErrors.empty())
    {
        cout << "No semantic errors found.\n";
        return;
    }

    for (const auto &error : semanticErrors)
    {
        cout << error << '\n';
    }
}

void printInterpreterOutput(const vector<string> &output)
{
    cout << "\n" << string(70, '=') << '\n';
    cout << "INTERPRETER OUTPUT\n";
    cout << string(70, '=') << '\n';
    if (output.empty())
    {
        cout << "<no output>\n";
        return;
    }

    for (const auto &line : output)
    {
        cout << line << '\n';
    }
}

void printRuntimeErrors(const vector<string> &runtimeErrors)
{
    cout << "\n" << string(70, '=') << '\n';
    cout << "RUNTIME ERRORS\n";
    cout << string(70, '=') << '\n';
    if (runtimeErrors.empty())
    {
        cout << "No runtime errors found.\n";
        return;
    }

    for (const auto &error : runtimeErrors)
    {
        cout << error << '\n';
    }
}
```